

C/C++

```
#include <Wire.h>
#include <hd44780.h>
#include <hd44780ioClass/hd44780_I2Cexp.h>
#include <SPI.h>
#include <MFRC522.h>
#include <HardwareSerial.h>
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <HTTPClient.h>
#include <WiFiManager.h>
#include <Preferences.h>
#include <ArduinoJson.h>
#include <PubSubClient.h>
#include "time.h"
```

```
//
```

```
=====
```

```
// 1. HARDWARE
```

```
//
```

```
=====
```

```
// LCDs
```

```
#define PIN_I2C_SDA 21
```

```
#define PIN_I2C_SCL 22
```

```
#define LCD_ADDR_MAIN 0x27
```

```
#define LCD_ADDR_SEC 0x26
```

```
// RFID
```

```
#define PIN_RFID_SS 5
```

```
#define PIN_RFID_SCK 18
```

```
#define PIN_RFID_MISO 19
```

```
#define PIN_RFID_MOSI 23
```

```
#define PIN_RFID_RST 4
```

```
// BALANÇA
```

```
#define PIN_SCALE_RX 17
```

```
#define PIN_SCALE_TX 16
```

```
// Velocidade definida pelo Scanner
```

```

// BOTÕES
#define PIN_BTN_CONFIRMAR 25
#define PIN_BTN_CANCELAR 26
#define PIN_BTN_FINALIZAR 27
#define PIN_BTN_SETA_CIMA 14
#define PIN_BTN_SETA_BAIXO 32
#define PIN_LED_VERDE 12
#define PIN_LED_VERMELHO 13

//
=====

// 2. OBJETOS
//
=====

hd44780_I2Cexp lcd_main(LCD_ADDR_MAIN);
hd44780_I2Cexp lcd_sec(LCD_ADDR_SEC);

MFRC522 rfid(PIN_RFID_SS, PIN_RFID_RST);
HardwareSerial ScaleSerial(2);
Preferences preferences;
WiFiClientSecure espClient;
PubSubClient mqttClient(espClient);
HTTPClient http;

//
=====

// 3. VARIÁVEIS GLOBAIS
//
=====

const char* URL_BASE =
"http://212.85.19.168:3000/api/v1/placa/pesagem";
const char* AP_PASSWORD_PROTECTION = "Admin1234";

// MQTT
char mqtt_broker[100];
char mqtt_port[6];

```

```

char mqtt_user[40];
char mqtt_pass[40];
const char* DEFAULT_BROKER =
"6df66d562a2e4bbdb0555f70df83a04d.s1.eu.hivemq.cloud";
const char* DEFAULT_PORT = "8883";
const char* DEFAULT_USER = "Samuel";
const char* DEFAULT_PASS = "Telecom25";

String MQTT_CLIENT_ID;
String MQTT_TOPIC_STATUS;
String payloadOnline;
String payloadOffline;

const char* NTP_SERVER = "pool.ntp.org";
const long GMT_OFFSET_SEC = -3 * 3600;

enum SistemaState {
    STATE_BOOT, STATE_WIFI_CONFIG, STATE_WAIT_FOR_CARD,
    STATE_CARD_AUTH,
    STATE_SHOW_CARD_ERROR, STATE_CONFIRM_USER,
    STATE_SELECT_PACKAGE,
    STATE_SELECT_WEIGHT, STATE_CONFIRM_WEIGH_START,
    STATE_WEIGHING_LOOP,
    STATE_WEIGHING_SUCCESS, STATE_WEIGHING_ERROR,
    STATE_SENDING_REPORT,
    STATE_REPORT_ERROR
};

SistemaState currentState = STATE_BOOT;

String board_position = "";
String last_card_id = "";
String employee_name = "";
JsonDocument employee_data;
String globalCookie = "";

int menu_index_pkg = 0;
int menu_index_wgt = 0;

```

```

struct SelectedPackage {
    String name;
    float weight;
    float tolerance;
} selected_package;

float total_weight_session = 0.0;
int package_count_session = 0;
unsigned long session_start_time_epoch = 0;
unsigned long session_end_time_epoch = 0;

bool shouldSaveConfig = false;

// Estrutura para o Scanner
struct SerialOption {
    long baud;
    uint32_t config;
    const char* label;
};

// Lista de tentativas comuns
SerialOption scaleOptions[] = {
    {9600, SERIAL_8N1, "9600 8N1"},
    {4800, SERIAL_8N1, "4800 8N1"},
    {4800, SERIAL_7E1, "4800 7E1"},
    {2400, SERIAL_8N1, "2400 8N1"},
    {2400, SERIAL_7E1, "2400 7E1"}
};

//
=====
// 4. FUNÇÕES AUXILIARES
//
=====

void lcdMainPrint(String l1, String l2) {
    lcd_main.clear();

```

```

    lcd_main.setCursor(0, 0); lcd_main.print(l1.substring(0,
16));
    lcd_main.setCursor(0, 1); lcd_main.print(l2.substring(0,
16));
}

void lcdSecPrint(String l1, String l2) {
    lcd_sec.clear();
    lcd_sec.setCursor(0, 0); lcd_sec.print(l1.substring(0, 16));
    lcd_sec.setCursor(0, 1); lcd_sec.print(l2.substring(0, 16));
}

void setLeds(bool verde, bool vermelho) {
    digitalWrite(PIN_LED_VERDE, verde ? HIGH : LOW);
    digitalWrite(PIN_LED_VERMELHO, vermelho ? HIGH : LOW);
}

String readRFID() {
    if (rfid.PICC_IsNewCardPresent() &&
rfid.PICC_ReadCardSerial()) {
        unsigned long long decNormal = 0;
        for (int i = 0; i < rfid.uid.size; i++) {
            decNormal = (decNormal << 8) | rfid.uid.uidByte[i];
        }
        rfid.PICC_HaltA();
        rfid.PCD_StopCrypto1();
        char buffer[25];
        sprintf(buffer, "%llu", decNormal);
        return String(buffer);
    }
    return "";
}

unsigned long getTimestampEpoch() {
    time_t now; time(&now); return (unsigned long)now;
}

unsigned long lastDebounceTime = 0;
const unsigned long debounceDelay = 250;

```

```

bool readButton(int pin) {
    if (digitalRead(pin) == LOW) {
        if ((millis() - lastDebounceTime) > debounceDelay) {
            lastDebounceTime = millis();
            return true;
        }
    }
    return false;
}

String getChipID() {
    uint64_t chipid = ESP.getEfuseMac();
    char chipIdStr[13];
    snprintf(chipIdStr, 13, "%04X%08X", (uint16_t)(chipid >>
32), (uint32_t)chipid);
    return String(chipIdStr);
}

// === SCANNER DE BALANÇA ===
void autoConfigScale() {
    lcdMainPrint("BUSCANDO", "BALANCA...");
    lcdSecPrint("Aguarde...", "Config Auto");

    Serial.println("\n--- INICIANDO SCANNER DE BALANÇA ---");
    bool found = false;

    pinMode(PIN_SCALE_RX, INPUT_PULLUP);

    int numOptions = sizeof(scaleOptions) /
sizeof(scaleOptions[0]);

    for (int i = 0; i < numOptions; i++) {
        Serial.printf("Tentando: %s... ", scaleOptions[i].label);
        lcdSecPrint("Tentando:", scaleOptions[i].label);

        ScaleSerial.begin(scaleOptions[i].baud,
scaleOptions[i].config, PIN_SCALE_RX, PIN_SCALE_TX);

        delay(100);
    }
}

```

```

while(ScaleSerial.available()) ScaleSerial.read();

unsigned long start = millis();
while (millis() - start < 1500) {
    if (ScaleSerial.available()) {
        char c = ScaleSerial.read();
        if (isdigit(c)) {
            found = true;
            Serial.println("DETECTADO!");
            lcdMainPrint("BALANCA OK!", scaleOptions[i].label);
            delay(1000);
            return;
        }
    }
}

Serial.println("Sem resposta.");
ScaleSerial.end();
delay(100);
}

Serial.println("Nenhuma balança detectada. Usando padrão.");
lcdMainPrint("FALHA DETECCAO", "Usando Padrao");
ScaleSerial.begin(9600, SERIAL_8N1, PIN_SCALE_RX,
PIN_SCALE_TX);
delay(1000);
}

// === LEITURA DE BALANÇA ===
float getWeightFromScale() {
    if (ScaleSerial.available() > 0) {
        String response = ScaleSerial.readStringUntil('\n');

        String bufferNumerico = "";
        bool temPonto = false;

        for (int i = 0; i < response.length(); i++) {
            char c = response[i];
            if (isdigit(c)) {

```

```

        bufferNumerico += c;
    } else if (c == '.' || c == ',') {
        if (!temPonto) {
            bufferNumerico += '.';
            temPonto = true;
        }
    } else if (c == '-') {
        // Ignora negativos
    }
}

if (bufferNumerico.length() > 0) {
    float w = bufferNumerico.toFloat();
    return w;
}
return -1.0;
}

void tareScaleSerial() {
    lcdMainPrint("TARANDO...", "Aguarde...");
    lcdSecPrint("Nao toque na", "balanca");
    ScaleSerial.write('T');
    ScaleSerial.write('Z');
    delay(1500);
}

void saveConfigCallback () {
    Serial.println("Config salva.");
    shouldSaveConfig = true;
}

void setupWiFiManager();
void setupMQTT();

// Estados
void state_waitForCard();
void state_cardAuth();
void state_showCardError();

```

```

void state_confirmUser();
void state_selectPackage();
void state_selectWeight();
void state_confirmWeighStart();
void state_weighingLoop();
void state_weighingSuccess();
void state_weighingError();
void state_sendingReport();
void state_reportError();

//
=====

// 5. SETUP
//
=====

void setup() {
  Serial.begin(115200);
  Serial.println("\n\n--- INICIANDO SISTEMA ---");

  pinMode(PIN_BTN_CONFIRMAR, INPUT);
  pinMode(PIN_BTN_CANCELAR, INPUT);
  pinMode(PIN_BTN_SETA_CIMA, INPUT);
  pinMode(PIN_BTN_SETA_BAIXO, INPUT);
  pinMode(PIN_BTN_FINALIZAR, INPUT);

  pinMode(PIN_LED_VERMELHO, OUTPUT);
  pinMode(PIN_LED_VERDE, OUTPUT);
  digitalWrite(PIN_LED_VERDE, HIGH); delay(500);
  digitalWrite(PIN_LED_VERDE, LOW);

  Wire.begin(PIN_I2C_SDA, PIN_I2C_SCL);
  Wire.setClock(100000);

  lcd_main.begin(16, 2);
  lcd_sec.begin(16, 2);

  lcdMainPrint("INICIANDO...", "Carregando...");

```

```

    SPI.begin(PIN_RFID_SCK, PIN_RFID_MISO, PIN_RFID_MOSI,
PIN_RFID_SS);
    delay(500); rfid.PCD_Init();

    // SCANNER REMOVIDO DO SETUP. AGORA É CHAMADO NA PESAGEM.

    preferences.begin("balanca-cfg", false);

    board_position = preferences.getString("bancada", "1");
    String s_broker = preferences.getString("mqtt_broker",
DEFAULT_BROKER);
    String s_port = preferences.getString("mqtt_port",
DEFAULT_PORT);
    String s_user = preferences.getString("mqtt_user",
DEFAULT_USER);
    String s_pass = preferences.getString("mqtt_pass",
DEFAULT_PASS);

    s_broker.toCharArray(mqtt_broker, 100);
    s_port.toCharArray(mqtt_port, 6);
    s_user.toCharArray(mqtt_user, 40);
    s_pass.toCharArray(mqtt_pass, 40);
    preferences.end();

    setupWiFiManager();
    espClient.setInsecure();
    configTime(-3 * 3600, 0, NTP_SERVER);
    setupMQTT();

    const char *headerKeys[] = {"Set-Cookie", "Cookie"};
    http.collectHeaders(headerKeys, 2);

    String hardwareID = getChipID();

    Serial.println("\n=====");
    Serial.print("HARDWARE ID: "); Serial.println(hardwareID);
    Serial.print("ID BANCADA (Configurada): ");
    Serial.println(board_position);

```

```

Serial.println("=====\\n");

  lcdMainPrint("ID PLACA:", hardwareID);
  lcdSecPrint("ID: " + board_position, "Aguarde");
  digitalWrite(PIN_LED_VERDE, HIGH); delay(3000);
  digitalWrite(PIN_LED_VERDE, LOW);

  currentState = STATE_WAIT_FOR_CARD;
}

void setupWiFiManager() {
  WiFiManager wm;
  wm.setSaveConfigCallback(saveConfigCallback);
  wm.setConnectTimeout(40);

  lcdMainPrint("CONF. WIFI", "Busque a Rede:");
  lcdSecPrint("ESP32-Pesagem", "Buscando...");

  WiFiManagerParameter custom_board_pos("bancada", "ID
Bancada", board_position.c_str(), 40);
  WiFiManagerParameter custom_mqtt_server("server", "MQTT
Broker", mqtt_broker, 100);
  WiFiManagerParameter custom_mqtt_port("port", "MQTT Port",
mqtt_port, 6);
  WiFiManagerParameter custom_mqtt_user("user", "MQTT User",
mqtt_user, 40);
  WiFiManagerParameter custom_mqtt_pass("pass", "MQTT Pass",
mqtt_pass, 40);

  wm.addParameter(&custom_board_pos);
  wm.addParameter(&custom_mqtt_server);
  wm.addParameter(&custom_mqtt_port);
  wm.addParameter(&custom_mqtt_user);
  wm.addParameter(&custom_mqtt_pass);

  if (!wm.autoConnect("ESP32-Pesagem",
AP_PASSWORD_PROTECTION)) {
    Serial.println("Falha ao conectar. Reiniciando...");
  }
}

```

```

    lcdMainPrint("FALHA WIFI", "Reiniciando...");
    delay(2000);
    ESP.restart();
}

lcdMainPrint("WIFI CONECTADO!", "Configurando...");

if (shouldSaveConfig) {
    board_position = custom_board_pos.getValue();

    strcpy(mqtt_broker, custom_mqtt_server.getValue());
    strcpy(mqtt_port, custom_mqtt_port.getValue());
    strcpy(mqtt_user, custom_mqtt_user.getValue());
    strcpy(mqtt_pass, custom_mqtt_pass.getValue());

    preferences.begin("balanca-cfg", false);
    preferences.putString("bancada", board_position);
    preferences.putString("mqtt_broker", mqtt_broker);
    preferences.putString("mqtt_port", mqtt_port);
    preferences.putString("mqtt_user", mqtt_user);
    preferences.putString("mqtt_pass", mqtt_pass);
    preferences.end();
}

}

void setupMQTT() {
    String hardwareID = getChipID();

    MQTT_CLIENT_ID = hardwareID;

    MQTT_TOPIC_STATUS = "balanca/status";

    payloadOnline = "{\"id\":\"" + hardwareID + "\",\n"
    + "status\":\"online\"}";
    payloadOffline = "{\"id\":\"" + hardwareID + "\",\n"
    + "status\":\"offline\"}";

    int port_int = atoi(mqtt_port);
    mqttClient.setServer(mqtt_broker, port_int);

```

```

mqttClient.setBufferSize(1024);
mqttClient.setKeepAlive(20);

if (mqttClient.connect(MQTT_CLIENT_ID.c_str(), mqtt_user,
mqtt_pass,
MQTT_TOPIC_STATUS.c_str(), 1, true,
payloadOffline.c_str())) {
    mqttClient.publish(MQTT_TOPIC_STATUS.c_str(),
payloadOnline.c_str(), true);
}
}

void loop() {
    if (WiFi.status() == WL_CONNECTED &&
!mqttClient.connected()) {
        static unsigned long lastReconnectAttempt = 0;
        if (millis() - lastReconnectAttempt > 5000) {
            lastReconnectAttempt = millis();
            if (mqttClient.connect(MQTT_CLIENT_ID.c_str(),
mqtt_user, mqtt_pass,
MQTT_TOPIC_STATUS.c_str(), 1,
true, payloadOffline.c_str())) {
                mqttClient.publish(MQTT_TOPIC_STATUS.c_str(),
payloadOnline.c_str(), true);
            }
        }
    }
    mqttClient.loop();

    switch (currentState) {
        case STATE_WAIT_FOR_CARD:      state_waitForCard();
break;
        case STATE_CARD_AUTH:          state_cardAuth(); break;
        case STATE_SHOW_CARD_ERROR:    state_showCardError();
break;
        case STATE_CONFIRM_USER:        state_confirmUser();
break;
        case STATE_SELECT_PACKAGE:      state_selectPackage();
break;
    }
}

```

```

        case STATE_SELECT_WEIGHT:      state_selectWeight();
break;
        case STATE_CONFIRM_WEIGH_START: state_confirmWeighStart();
break;
        case STATE_WEIGHING_LOOP:      state_weighingLoop();
break;
        case STATE_WEIGHING_SUCCESS:   state_weighingSuccess();
break;
        case STATE_WEIGHING_ERROR:     state_weighingError();
break;
        case STATE_SENDING_REPORT:     state_sendingReport();
break;
        case STATE_REPORT_ERROR:       state_reportError();
break;
        default: break;
    }
}

void state_waitForCard() {
    lcdMainPrint("APROXIME CARTAO", "P/ INICIAR...");
    lcdSecPrint("Local: " + board_position, "Livre");
    setLeds(false, false);

    String card = readRFID();
    if (card != "") {
        last_card_id = card;
        lcdMainPrint("LENDO ID:", last_card_id);
        lcdSecPrint("Verificando...", "Aguarde");
        digitalWrite(PIN_LED_VERDE, HIGH); delay(100);
digitalWrite(PIN_LED_VERDE, LOW);
        delay(3000);

        currentState = STATE_CARD_AUTH;
    }
}

void state_cardAuth() {
    Serial.println("\n--- AUTH (GET) ---");
    if (WiFi.status() != WL_CONNECTED) {

```

```

    lcdMainPrint("SEM WIFI", "ID:" + last_card_id);
    delay(3000);
    currentState = STATE_WAIT_FOR_CARD; return;
}
last_card_id.trim();
String fullUrl = String(URL_BASE) + "/" + last_card_id;

Serial.println("URL: " + fullUrl);

http.setTimeout(10000);
http.begin(fullUrl);
http.addHeader("Accept", "application/json");
http.addHeader("User-Agent", "ESP32");

int httpCode = http.GET();
Serial.print("HTTP Code: "); Serial.println(httpCode);

if (httpCode == 200) {
    String payload = http.getString();
    Serial.println("Resp: " + payload);

    if (http.hasHeader("Set-Cookie")) {
        globalCookie = http.header("Set-Cookie");
        Serial.println("Cookie capturado: " + globalCookie);
    }

    JsonDocument docLocal;
    DeserializationError error = deserializeJson(docLocal,
payload);
    employee_name = "";
    if (!error) {
        if(docLocal.containsKey("employee_name")) {
            const char* n = docLocal["employee_name"];
            if(n && strlen(n) > 0) employee_name = String(n);
            else employee_name = "Funcionario";
        } else { employee_name = "Funcionario"; }
        employee_data.clear();
        employee_data = docLocal;
    }
}

```

```

        JSONArray packages =
employee_data["packages"].as<JSONArray>();
        if (packages.size() > 0) { currentState =
STATE_CONFIRM_USER; }
        else { lcdMainPrint("SEM PACOTES", "ID:" +
last_card_id); delay(3000); currentState =
STATE_WAIT_FOR_CARD; }
        } else { lcdMainPrint("ERRO JSON", "Tente Novamente");
delay(3000); currentState = STATE_WAIT_FOR_CARD; }
        } else if (httpCode == 404) { lcdMainPrint("NAO
RECONHECIDO", "Verifique ID"); delay(4000); currentState =
STATE_WAIT_FOR_CARD;
        } else { lcdMainPrint("ERRO: " + String(httpCode), "Tente
Novamente"); setLeds(false, true); delay(4000); currentState =
STATE_WAIT_FOR_CARD; }
        http.end();
    }

void state_showCardError() { currentState =
STATE_WAIT_FOR_CARD; }
void state_confirmUser() {
    lcdMainPrint("Ola,", employee_name);
    lcdSecPrint("1:CONFIRMAR", "2:CANCELAR");
    if (readButton(PIN_BTN_CONFIRMAR)) { menu_index_pkg = 0;
currentState = STATE_SELECT_PACKAGE; delay(300); }
    if (readButton(PIN_BTN_CANCELAR)) { currentState =
STATE_WAIT_FOR_CARD; delay(300); }
}

void state_selectPackage() {
    JSONArray packages =
employee_data["packages"].as<JSONArray>();
    int num_packages = packages.size();
    String pkgName =
packages[menu_index_pkg]["name"].as<String>();
    lcdMainPrint("PCT:", pkgName);
    lcdSecPrint(String(menu_index_pkg + 1) + "/" +
String(num_packages) + " NAV:Setas", "1:OK 2:VOLT");
}

```

```

    if (readButton(PIN_BTN_SETA_CIMA)) { menu_index_pkg =
(menu_index_pkg - 1 < 0) ? num_packages - 1 : menu_index_pkg -
1; delay(250); }
    if (readButton(PIN_BTN_SETA_BAIXO)) { menu_index_pkg =
(menu_index_pkg + 1 >= num_packages) ? 0 : menu_index_pkg + 1;
delay(250); }
    if (readButton(PIN_BTN_CONFIRMAR)) {
        selected_package.name = pkgName;
        selected_package.tolerance =
packages[menu_index_pkg]["tolerance"].as<float>();
        menu_index_wgt = 0; currentState = STATE_SELECT_WEIGHT;
delay(300);
    }
    if (readButton(PIN_BTN_CANCELAR)) { currentState =
STATE_WAIT_FOR_CARD; delay(300); }
}

void state_selectWeight() {
    JSONArray weights =
employee_data["packages"][menu_index_pkg]["weight"].as<JsonArr
ay>();
    int num_weights = weights.size();
    float wVal = weights[menu_index_wgt].as<float>();

    lcdMainPrint("PESO REF:", String(wVal, 3) + " kg");

    lcdSecPrint("1:SELECIONAR", "2:VOLTAR");
    if (readButton(PIN_BTN_SETA_CIMA)) { menu_index_wgt =
(menu_index_wgt - 1 < 0) ? num_weights - 1 : menu_index_wgt -
1; delay(250); }
    if (readButton(PIN_BTN_SETA_BAIXO)) { menu_index_wgt =
(menu_index_wgt + 1 >= num_weights) ? 0 : menu_index_wgt + 1;
delay(250); }
    if (readButton(PIN_BTN_CONFIRMAR)) { selected_package.weight
= wVal; currentState = STATE_CONFIRM_WEIGH_START; delay(300);
    }
    if (readButton(PIN_BTN_CANCELAR)) { currentState =
STATE_SELECT_PACKAGE; delay(300); }
}

```

```

void state_confirmWeighStart() {
    lcdMainPrint("INICIAR SEQ?", String(selected_package.weight,
3) + "kg");
    lcdSecPrint("1:SIM | 2:NAO", "");

    if (readButton(PIN_BTN_CONFIRMAR)) {
        session_start_time_epoch = getTimestampEpoch();
        total_weight_session = 0.0; package_count_session = 0;

        autoConfigScale();
        tareScaleSerial();

        currentState = STATE_WEIGHING_LOOP; delay(300);
    }
    if (readButton(PIN_BTN_CANCELAR)) { currentState =
STATE_SELECT_WEIGHT; delay(300); }
}

void state_weighingLoop() {
    float current_weight = getWeightFromScale();
    static float display_weight = 0.0;
    if (current_weight >= 0.0) display_weight = current_weight;

    lcdMainPrint("PESO: " + String(display_weight, 3) + "kg",
"META: " + String(selected_package.weight, 3) + "kg");

    lcdSecPrint("QTD: " + String(package_count_session) + " |
Tot:" + String((int)total_weight_session), "1:GRAVAR 3:FIM");
    setLeds(false, false);
    if (readButton(PIN_BTN_CONFIRMAR)) {
        float min_w = selected_package.weight;
        float max_w = selected_package.weight +
selected_package.tolerance;

        if (display_weight >= min_w && display_weight <= max_w) {

```

```

        total_weight_session += display_weight;
package_count_session++; currentState =
STATE_WEIGHING_SUCCESS;
    } else { currentState = STATE_WEIGHING_ERROR; }
    delay(500);
}
    if (readButton(PIN_BTN_FINALIZAR)) { session_end_time_epoch
= getTimestampEpoch(); currentState = STATE_SENDING_REPORT;
delay(300); }
}

void state_weighingSuccess() {
    lcdMainPrint("SUCESSO!", "Peso Registrado");
    lcdSecPrint("QTD Total: " + String(package_count_session),
"Remova o Peso");
    setLeds(true, false); delay(3000);
    unsigned long s = millis();
    while (getWeightFromScale() > 0.05 && (millis() - s < 5000))
{ lcdSecPrint("Remova o Peso", "da Balanca..."); delay(100); }
    currentState = STATE_WEIGHING_LOOP;
}

void state_weighingError() {
    lcdMainPrint("ERRO: PESO", "FORA TOLERANCIA");
    lcdSecPrint("Esperado: " + String(selected_package.weight,
3), "Tol: " + String(selected_package.tolerance, 3));

    setLeds(false, true); delay(4000); currentState =
STATE_WEIGHING_LOOP;
}

void state_sendingReport() {
    lcdMainPrint("ENVIANDO DADOS", "Aguarde...");
    JsonDocument doc_rep;

    String hardwareID = getChipID();
    doc_rep["terminal_id"] = hardwareID;
    Serial.print("Terminal ID: "); Serial.println(hardwareID);

```

```

doc_rep["employee_id"]    = last_card_id;
doc_rep["employee_name"] = employee_name;

JsonObject pkgObj = doc_rep.createNestedObject("package");

pkgObj["name"]      = selected_package.name;
pkgObj["weight"]    = selected_package.weight;
pkgObj["quantity"]  = package_count_session;
pkgObj["tolerance"] = selected_package.tolerance;

doc_rep["total_weight"] = total_weight_session;

if (session_end_time_epoch == 0) {
    session_end_time_epoch = getTimestampEpoch();
}

char timeBuff[35];

time_t t_start = (time_t)session_start_time_epoch;
struct tm *tm_start = localtime(&t_start);
strftime(timeBuff, sizeof(timeBuff), "%Y-%m-%dT%H:%M:%S",
tm_start);
doc_rep["timestamp_begin"] = String(timeBuff);

time_t t_end = (time_t)session_end_time_epoch;
struct tm *tm_end = localtime(&t_end);
strftime(timeBuff, sizeof(timeBuff), "%Y-%m-%dT%H:%M:%S",
tm_end);
doc_rep["timestamp_end"] = String(timeBuff);

String jsonReq;
serializeJson(doc_rep, jsonReq);

String urlFinal = String(URL_BASE);

Serial.println("\n--- ENVIANDO RELATORIO ---");
Serial.print("URL POST: "); Serial.println(urlFinal);
Serial.print("JSON: "); Serial.println(jsonReq);

```

```

http.setTimeout(10000);
http.begin(urlFinal);
http.addHeader("Content-Type", "application/json");
http.addHeader("User-Agent", "ESP32");

if(globalCookie.length() > 0) {
    http.addHeader("Cookie", globalCookie);
}

int httpCode = http.POST(jsonReq);
Serial.print("HTTP Code: "); Serial.println(httpCode);

if (httpCode == 200 || httpCode == 201) {
    lcdMainPrint("DADOS ENVIADOS", "SUCESSO (" +
String(httpCode) + ")");
    delay(2000);
} else {
    lcdMainPrint("FALHA ENVIO", "Erro: " + String(httpCode));
    if(httpCode > 0) Serial.println("Resposta: " +
http.getString());
    delay(4000);
}

session_end_time_epoch = 0;
currentState = STATE_WAIT_FOR_CARD;
http.end();
}

void state_reportError() { currentState = STATE_WAIT_FOR_CARD;
}

```